

StickyInvoc: Rethinking Task Models for High-throughput Workflows in the LLM Era

Thanh Son Phung, Douglas Thain

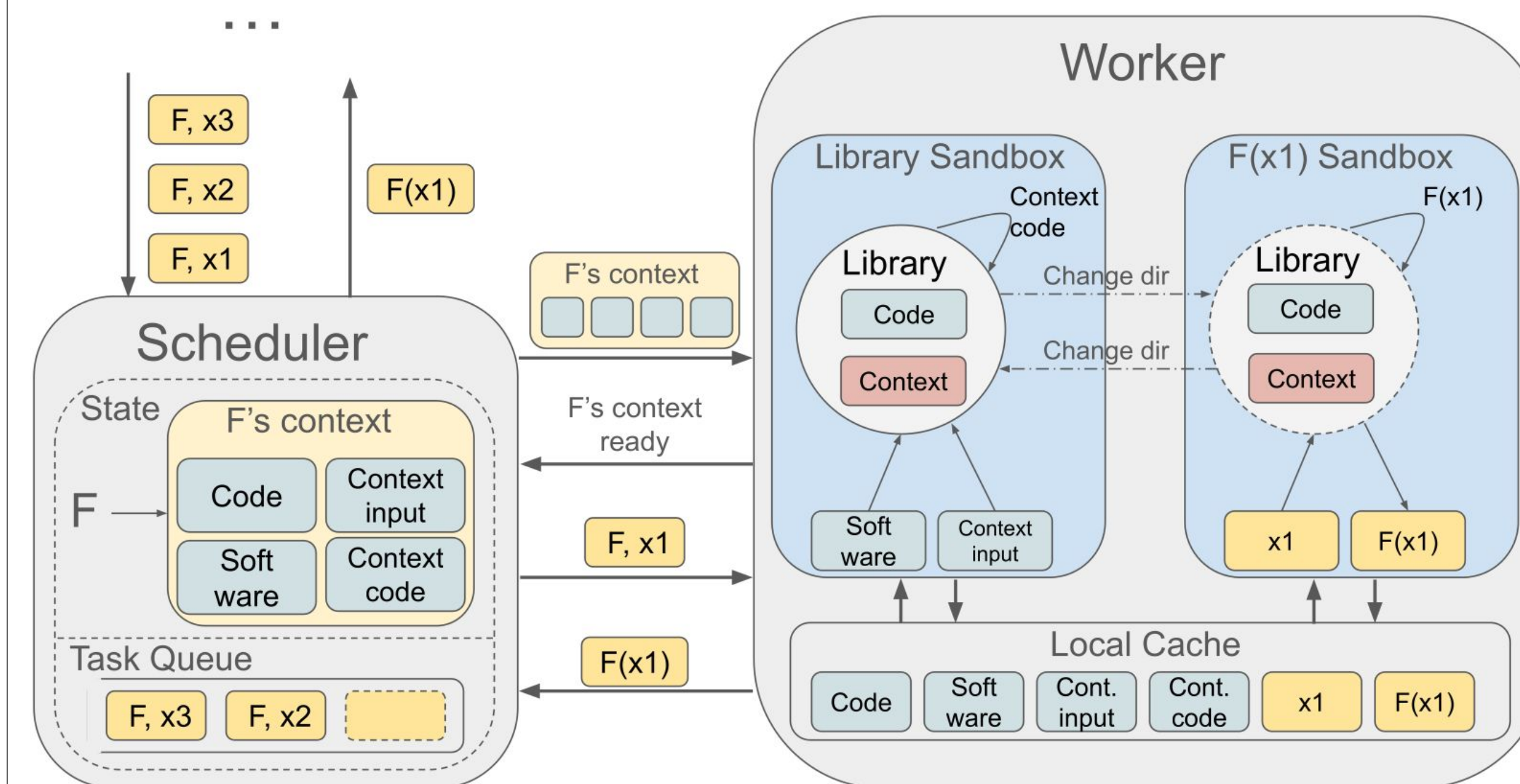
Cooperative Computing Lab
University of Notre Dame
{tphung, dthain}@nd.edu

Abstract

Executing large-scale, high-throughput lightweight Large Language Model (LLM) inference workflows on conventional shared clusters leads to long job queues due to inefficient static resource allocation. While opportunistic GPU clusters offer a vast pool of compute, their transient nature makes them challenging to utilize, as the overhead of repeatedly initializing an inference job's state with billions of parameters after preemption is prohibitively high.

We introduce StickyInvoc, a symbiotic relationship between two new task models for high-throughput workflows. Specifically, a “sticky” task creates a persistent and inheritable state on a worker node from a user-provided template, while subsequent “invocation” tasks inherit this state and perform the actual computation without incurring the state creation overhead or destroying the state upon exit. This allows efficient reuse and rapid context transfer between inferences across intermittently available GPUs, virtually eliminating initialization costs. With StickyInvoc, a claim verification workflow achieves a 3.6x speedup on a stable testbed with 20 GPUs, and completes in just 784 seconds using 186 opportunistic GPUs.

Solution: Reusing LLM states with StickyInvoc



Subsequent inferences of F reuse the same LLM state in a remote worker's GPU, memory, and disk as managed by the **Library** process

Implementation: StickyInvoc in Parsl-TaskVine framework

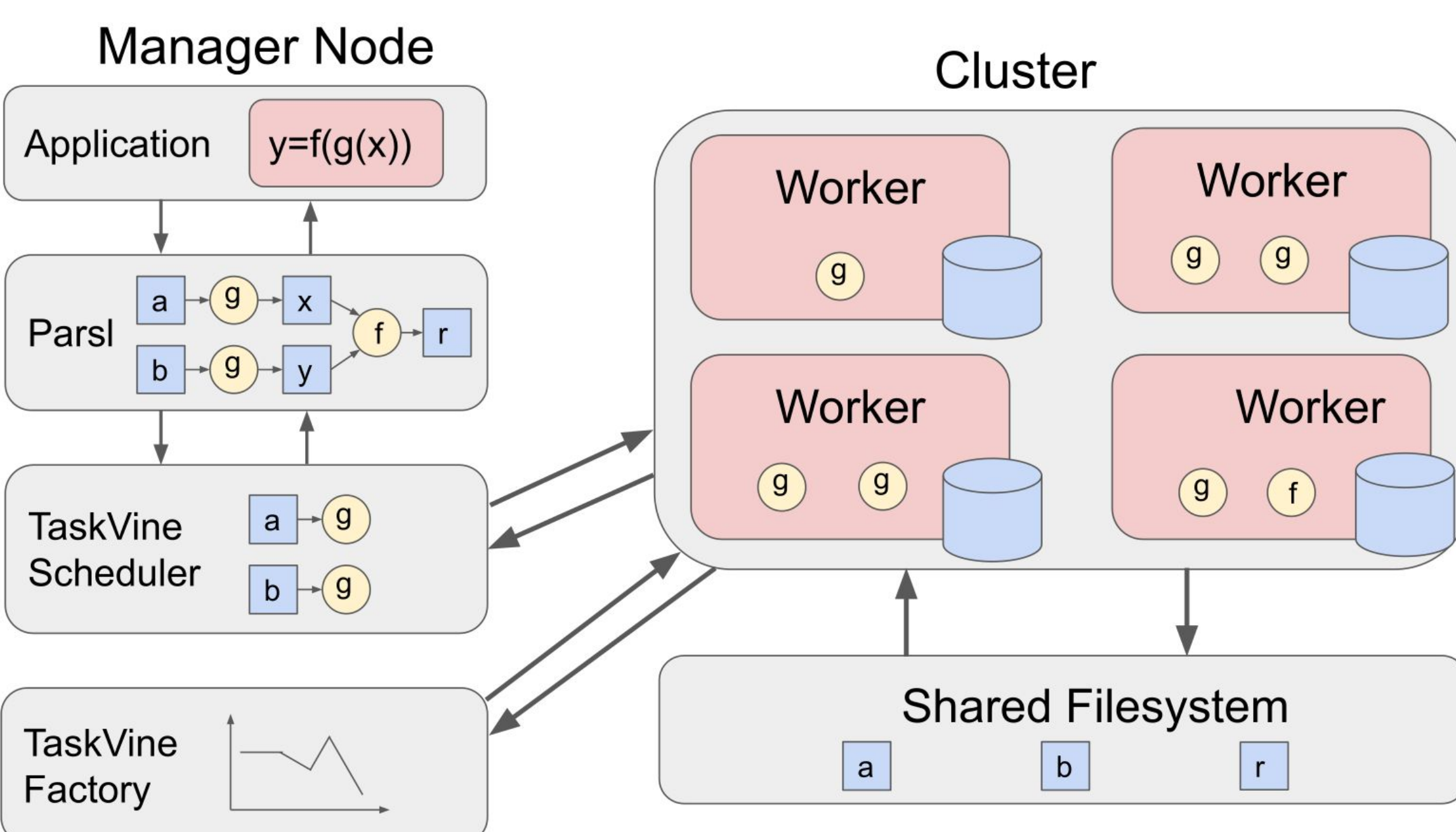
```
from parsl import python_app
def load_model(model_path):
    ...
    model = AutoModel.from_pretrained(model_path).to('gpu')
    return {'model': model}
```

```
@python_app
def infer_model(inputs, parsl_spec):
    from parsl import load_variable_from_serverless
    model = load_variable_from_serverless('model')
    outputs = [model.generate(input) for input in inputs]
    return outputs
```

```
model_path = ...
parsl_spec = {'context': [load_model, [model_path], {}]}
inputs = ...
results = infer_model(inputs, parsl_spec).result()
```

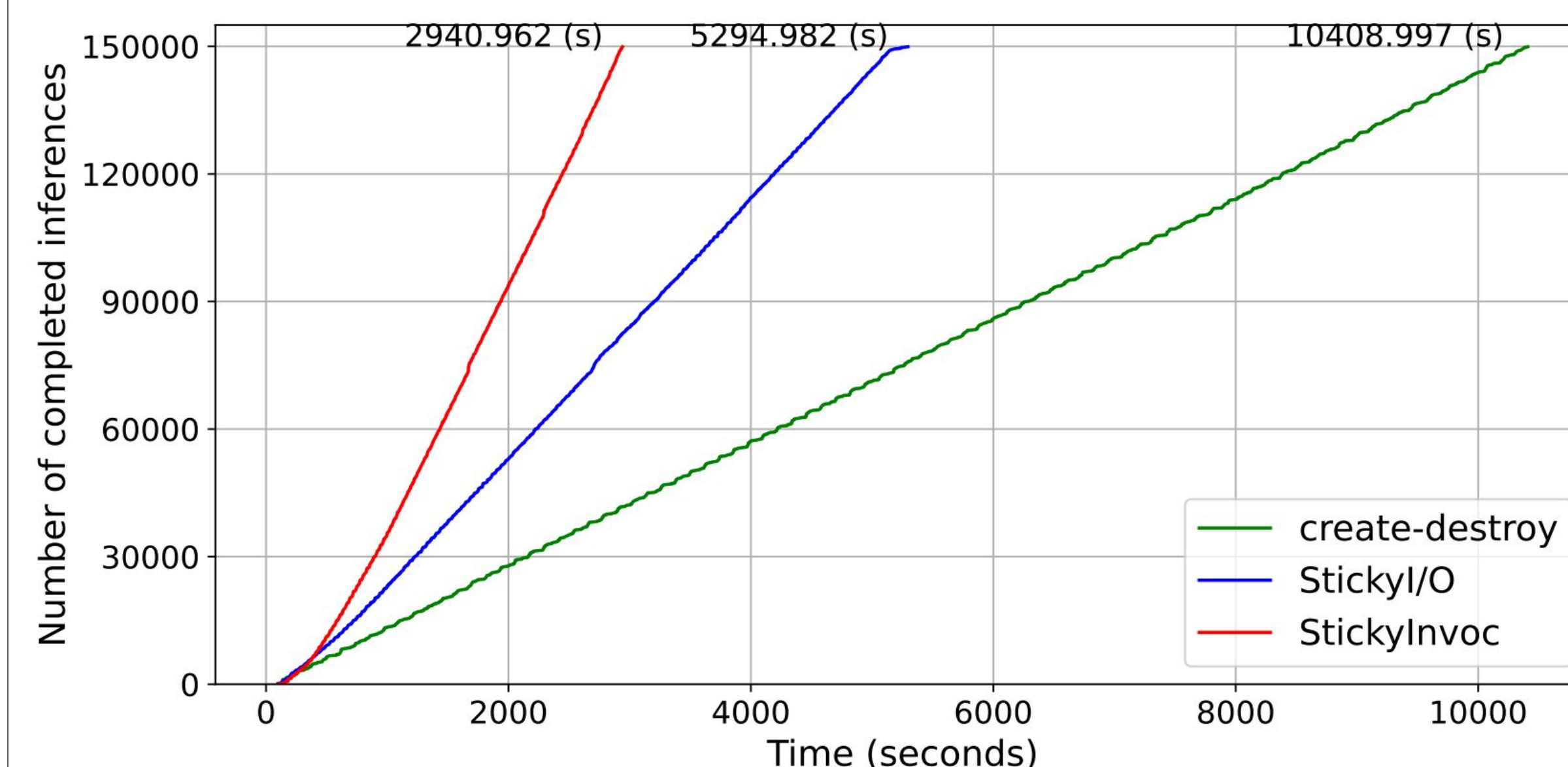
This sample shows how the inference context - a model state retained in a remote GPU - is **decoupled** from an inference execution, allowing its efficient reuse for subsequent inferences

Background: Parsl-TaskVine Parallel Framework

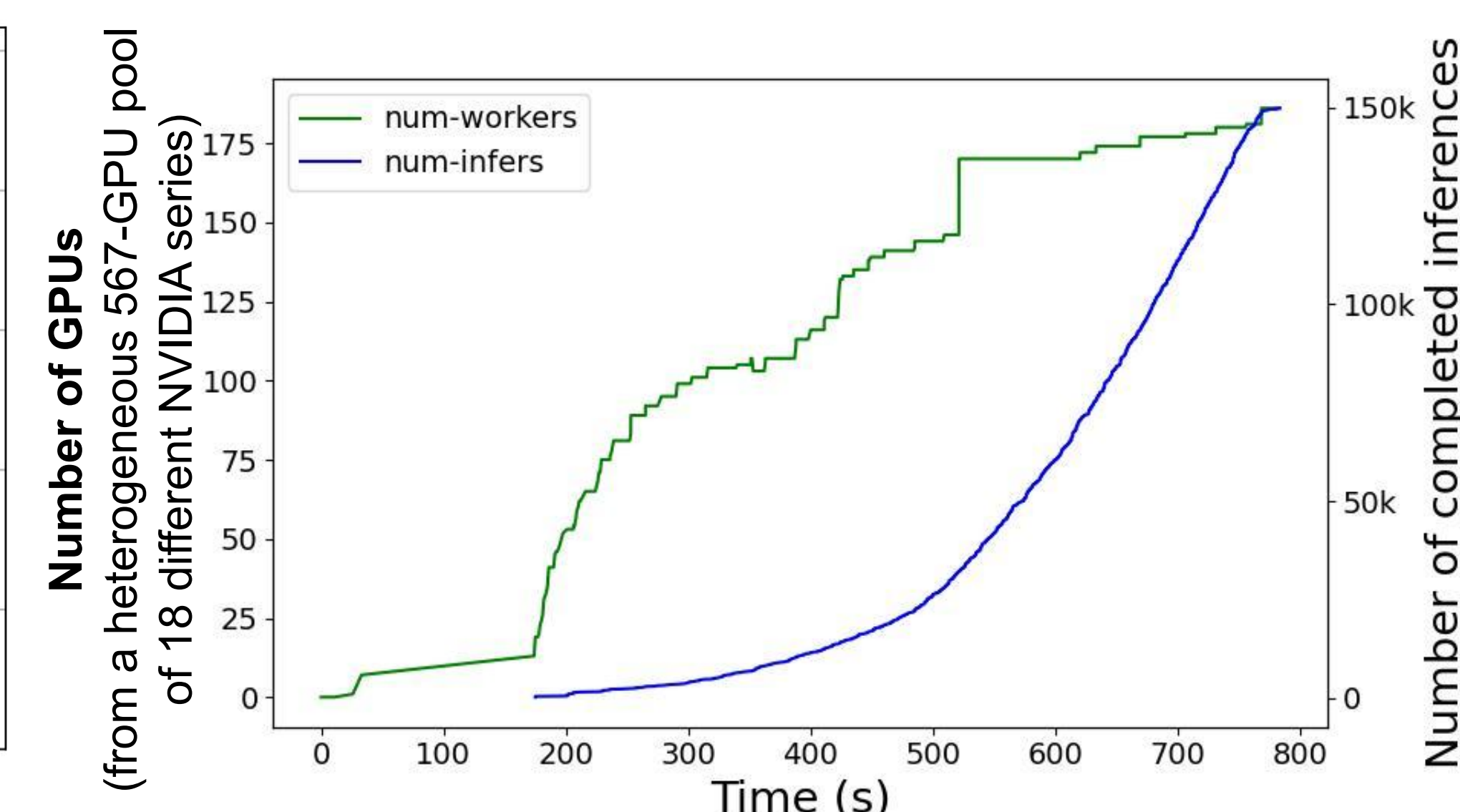


Functions generated from the application are passed to Parsl, which creates a DAG of functions and converts them into tasks for TaskVine Scheduler. Tasks are then scheduled to TaskVine Workers in the cluster, and TaskVine Factory moderates the number of workers as pre-specified.

Results: Claim verification workflow in 3 execution paradigms (left) and unconstrained GPU scaling (right)



Create-destroy: standard task execution with no state caching. StickyI/O: common data between tasks are cached on local disk. StickyInvoc: common state and data between tasks are cached on local memory and disk. All runs use the same set of 20 GPUs.



GPUs (i.e., workers) join the application's pool individually as soon as they are available, pushing the execution time of 150k inferences to 13 minutes